

제 7 주:

스택: 기본기능



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 7 주 실습]

스택 : 기본기능



□ 실습 목표

■ 이론적 관점

- 스택의 개념
- Java의 인터페이스 개념

■ 구현적 관점

- 스택을 배열 리스트로 구현하는 방법
- Generic class 사용법
- Interface 사용법

과제에서 해결할 문제



□ [프로그램 07]

- 키보드에서 문자를 반복 입력 받는다.
 - 한 번에 한 개의 문자를 입력 받는다.
- 입력의 종료 조건
 - '!' 키를 치면 더 이상 입력을 받지 않는다.
- 매번 한 개의 문자를 입력 받을 때 마다, 문자에 따라 정해진 일을 한다.
- Generic Class 를 사용해 본다.
- 인터페이스 (Interface) 를 사용하여 본다.

□ 이 과제에서 필요한 객체는?

- ApplicationController

- AppView

- Model

 - ArrayList<T>

□ 입출력

- 영문자 ('A' ~ 'Z', 'a' ~ 'z'): 스택에 삽입한다.
 - 다음과 같은 메시지를 내보낸다. (입력된 영문자가 'x' 라면)
 - ◆ "[Push] 삽입된 원소는 'x' 입니다."
 - 만일 스택이 full이면 다음과 같은 메시지를 내보낸다.
 - ◆ [Full] 스택이 꽉 차서 원소 'x' 는 삽입이 불가능합니다.
- '!': 스택의 원소를 모두 삭제하고, 프로그램을 종료한다.
 - 삭제할 때마다 다음과 같은 메시지를 내보낸다. (삭제된 원소가 'x' 라면)
 - ◆ "[End] 삭제된 원소는 'x' 입니다."

□ 입출력

- 숫자문자('0' ~ '9'): 해당 수 만큼 스택에서 삭제한다.
 - 먼저 다음 메시지를 내보낸다. ('3'이 입력되었다면)
 - ◆ [Pops] 3 개의 원소를 삭제하려고 합니다.
 - 매번 삭제될 때마다 다음과 같은 메시지를 내보낸다. (삭제된 원소가 'x' 라면)
 - ◆ "[Pops] 삭제된 원소는 'x' 입니다.
 - 삭제를 하려는데 스택이 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "[Empty] 스택에 더 이상 삭제할 원소가 없습니다."

- '-': 스택의 top 원소를 삭제한다.
 - 다음과 같은 메시지를 내보낸다. (Top 원소가 'r' 라면)
 - ◆ "[Pop] 삭제된 원소는 'r' 입니다.
 - 삭제를 하려는데 스택이 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "[Empty] 스택에 삭제할 원소가 없습니다."

□ 입출력

- '#': 스택의 길이를 다음과 같이 출력한다. (현재 스택에 5개의 원소가 있다면)
 - "[Size] 스택에는 현재 5개의 원소가 있습니다."
- '/': 스택의 내용을 Bottom 부터 Top 까지 다음과 같이 차례로 출력한다.
 - "[Stack] <Bottom> A b c d E f <Top>"
- '\': 스택의 내용을 Top부터 Bottom까지 다음과 같이 차례로 출력한다.
 - "[Stack] <Top> f E d c b A <Bottom>"
- '^': 스택의 top 원소의 값을 출력한다. 스택은 변하지 않는다. (스택의 top 원소가 'f'라면)
 - "[Top] Top 원소는 'f' 입니다."

□ 입출력

- 그 밖의 문자들: 다음과 같이 출력하고 무시한다.
 - "[Ignore] 의미 없는 문자가 입력되었습니다."

□ 출력의 예 [1]

```
> 프로그램을 시작합니다.
[스택 사용을 시작합니다.]
- 문자를 입력하시오: A
[Push] 삽입된 원소는 'A'입니다.
- 문자를 입력하시오: x
[Push] 삽입된 원소는 'x'입니다.
- 문자를 입력하시오: #
[Size] 스택에는 현재 2개의 원소가 있습니다.
- 문자를 입력하시오: h
[Push] 삽입된 원소는 'h'입니다.
- 문자를 입력하시오: /
[Stack] <Bottom> A x h <Top>
- 문자를 입력하시오: W
[Push] 삽입된 원소는 'W'입니다.
- 문자를 입력하시오: z
[Push] 삽입된 원소는 'z'입니다.
- 문자를 입력하시오: p
[Full] 스택이 꽉 차서 삽입이 불가능합니다.
- 문자를 입력하시오: \
[Stack] <Top> z W h x A <Bottom>
- 문자를 입력하시오: -
[Pop] 삭제된 원소는 'z'입니다.
- 문자를 입력하시오: ^
[Top] Top 원소는 'W'입니다.
```

```
- 문자를 입력하시오: 5
[Pop] 삭제된 원소는 'W'입니다.
[Pop] 삭제된 원소는 'h'입니다.
[Pop] 삭제된 원소는 'x'입니다.
[Pop] 삭제된 원소는 'A'입니다.
[Empty] 스택에 삭제할 원소가 없습니다.
- 문자를 입력하시오: B
[Push] 삽입된 원소는 'B'입니다.
- 문자를 입력하시오: e
[Push] 삽입된 원소는 'e'입니다.
- 문자를 입력하시오: !
[스택 입력을 종료합니다.]
[Pop] 삭제된 원소는 'e'입니다.
[Pop] 삭제된 원소는 'B'입니다.
--- 입력된 문자는 모두 14입니다.
--- 정상 처리된 문자는 모두 14입니다.
--- 무시된 문자는 모두 0입니다.
--- 삽입된 문자는 모두 7입니다.
> 프로그램을 종료합니다.
```

□ 출력의 예 [2]

```

> 프로그램을 시작합니다.
[스택 사용을 시작합니다.]
- 문자를 입력하시오: A
[Push] 삽입된 원소는 'A'입니다.
- 문자를 입력하시오: x
[Push] 삽입된 원소는 'x'입니다.
- 문자를 입력하시오: d
[Push] 삽입된 원소는 'd'입니다.
- 문자를 입력하시오: 6
[Pop] 삭제된 원소는 'd'입니다.
[Pop] 삭제된 원소는 'x'입니다.
[Pop] 삭제된 원소는 'A'입니다.
[Empty] 스택에 삭제할 원소가 없습니다.
- 문자를 입력하시오: &
[Error] 의미 없는 문자가 입력되었습니다.
- 문자를 입력하시오: !
[스택 사용을 종료합니다.]
...   .   .   입력된 문자는 총 5개 입니다.
...   .   .   정상 처리된 문자는 4개 입니다.
...   .   .   무시된 문자는 1개 입니다.
...   .   .   삽입된 문자는 3개 입니다.

> 프로그램을 종료합니다.

```

Generic Class



□ Why Generic Class ?[1]

- 창고 짓는 설계도를 그렸다:
 - 이 설계도가 컴퓨터 모니터 전용 창고 용도라면, 이 설계도로 지은 창고는 컴퓨터 모니터만 보관 가능.
- 그런데, 모니터나 키보드나 보관 방법이 다르지 않다면?
 - 모니터 보관용 창고 설계도로, 키보드 보관용 창고도 지을 수 있을 것이다.
 - 하나의 창고 설계도로 다양한 type의 객체를 보관할 수 있는 창고를 지을 수 있다.
- 예를 들어, class ArrayList 는 담아야 할 원소의 종류가 무엇이든지 그 기능은 동일한, 그러한 객체들의 설계도 이다.
- 그렇다면, 실제 생성된 ArrayList (창고)에 어떤 자료형의 원소를 담을 지는, 설계할 때 미리 결정해 놓을 것이 아니라, ArrayList 를 사용하는 시점에 결정할 수 있다면 편리할 것이다.

□ Why Generic Class ? [2]

- 만일, ArrayList 를 정의하는 시점에 담을 원소의 자료형을 고정시켜야만 된다면 ?
 - 그 ArrayList 는 고정된 한 type 의 객체 만을 담을 수 있는 ArrayList 가 될 것이다.
 - 또한 우리는 원소의 class 마다 별도의 ArrayList 를 만들어 사용해야 할 것이다.
 - ◆ ArrayListForStudent, ArrayListForFruits,
- 담아야 할 객체들의 다양한 type 마다,
 - 별도의 설계를 작성하는 것이 아니다.
 - 하나의 설계도만 작성하면 된다.
- 그러므로, generic class 를 사용하는 것은, 코드의 생산성을 높게 된다.
- 설계도가 유사하지만, 똑같지는 않다면?
 - 이 문제는, generic class 의 문제는 아니다.
 - (설계도) 의 상속 (inheritance)을 고려해 본다.

□ Generic Class:

- Java 는, 클래스를 설계할 때 그 내부에서 사용되는 자료형을 특정하지 않은 채로 통칭적인 (generic) 그래서 형식적인 식별자 (identifier) 를 사용할 수 있게 해 준다.
 - 이러한 형식적인 식별자로 정의되는 자료형을 generic type 이라고 한다.
 - 이 generic identifier 는 나중에 실제로 사용하는 시점에 실제로 존재하는 자료형 이름으로 대체가 된다.
- (예) 다음과 같은 식별자를 종종 사용한다. 그러나 식별자의 이름은 사용자가 임의로 정할 수 있다.
 - Class ArrayList <T>
 - ◆ Class "ArrayList"에서 원소의 type으로 사용되는 generic class "T"
 - Class Node <E>
 - ◆ Class "Node"에서 원소의 type으로 사용되는 generic class "E"
- 사용하는 관점에서, 동일한 class 일지라도 사용 시점에 generic type 이 다르게 주어진 class 들은 서로 다른 class 라고 할 수 있다:
 - Class "ArrayList<Student>" 와 class "ArrayList<Integer>" 는 서로 다른 class 이다.

□ Generic Class 를 배열 원소의 자료형으로 사용 [1]

```
public class ArrayList <T> {  
    private T[] _elements ;
```

```
    public ArrayList () {  
        this._elements = (T[]) new Object [100];  
    }  
}
```

- 객체를 생성하는 new 를 사용할 때에는 generic type 이 아닌, 반드시 구체적인 type 으로 언급되어야 한다.
- Class "ArrayList" 컴파일 하는 시점에, T 의 자료형은 결정되어 있지 않다. 따라서, new 에 T 를 사용하면 컴파일러는 오류를 내보낸다. (오류 메시지: "Cannot create a generic array of T")
- 보통 배열을 생성하는 이런 경우에는, T 대신에 최상위 Class 인 "Object" 를 사용한다. 즉, 아무 원소나 저장할 수 있는 배열을 만드는 것이다.
- 컴파일러는 생성된 배열이 _elements 의 자료형과 일치하지 않다고 여전히 오류 메시지 "Type mismatch: cannot convert from Object[] to T[]" 를 내보낸다.

□ Generic Class 를 배열 원소의 자료형으로 사용 [2]

```
public class ArrayList <T> {  
    private T[] _elements ;
```

```
    public ArrayList () {  
        this._elements = (T[]) new Object [100] ;  
    }  
}
```

- 자료형 불일치를 해결하기 위해서, new 앞에 "(T[])" 를 삽입하여 생성된 배열의 자료형을 강제로 T[] 로 변환시킨다.
- 이렇게 하여 컴파일 오류는 제거할 수 있다.
- 그렇지만, 나중에 사용자가 잘못 사용하여 자료형이 T 가 아닌 원소를 배열에 넣을 가능성이 있다. (앞에서 언급하였듯이 생성된 배열은 원소의 자료형이 "Object" 이므로 어떠한 자료형의 객체도 넣을 수 있다.)
- 이렇게 잘못 사용할 가능성이 있으므로 컴파일러는 경고 메시지를 내보낸다. (경고 메시지: "Type Safety: Unchecked cast from Object[] to T[]")

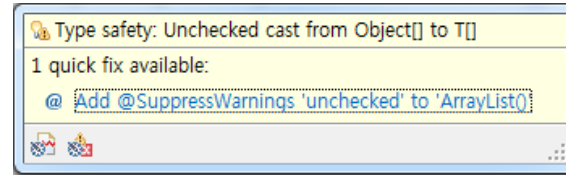
□ Generic Class 를 배열 원소의 자료형으로 사용 [3]

```
public class ArrayList <T> {  
    private T[] _elements ;  
  
    @SuppressWarnings("unchecked")  
    public ArrayList () {  
        this._elements = (T[]) new Object [100] ;  
    }  
}
```

- 프로그램 작성자는 이러한 강제적인 자료형 변환으로 인하여 발생할 수 있는 위험성을 인지하고 대처할 수 있는 방안을 가지고 있어야 한다.
- 이를테면, 배열에 들어가는 객체의 자료형은 언제나 T 임 스스로 보장할 수 있고, 또 관련 프로그램 코드를 그에 합당하게 작성할 것임을 스스로 보장할 수 있어야 한다.
- 그렇다면, 프로그램 작성자는, "컴파일러는 이러한 강제적인 자료형 변환을 걱정할 필요가 없으므로, 자료형 변환으로 인한 점검을 하지 않아도 된다" 고 컴파일러 알려 더 이상 경고 메시지가 통보되지 않게 할 수 있다.

□ Generic Class 를 배열 원소의 자료형으로 사용 [3]

```
public class ArrayList <T> {
    private T[] _elements ;
```



```
@SuppressWarnings("unchecked")
```

```
public ArrayList () {
    this._elements = (T[]) new Object [100] ;
}
```

오류를 클릭하면 이와 같은 창이 뜬다. 경고를 해결할 수 있는 방법을 선택하면 컴파일러가 자동으로 필요한 행위를 한다.

- 이러한 통보를 컴파일러에게 하기 위해, `@SuppressWarnings("unchecked")` 를 삽입한다. 프로그램 작성자는 이 통보 사항을 직접 입력하기 보다는 오류를 클릭하여 해결책을 선택하는 방법으로 처리하게 된다.

main()

□ main() 을 위한 class

```
public class DS1_07_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"

□ Class "AppController" 의 구현 [1]

```
public class AppController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private ArrayList<Character> _arrayStack ;  
    private int _inputChars ;           // 입력된 문자의 개수  
    private int _ignoredChars ;        // 무시된 문자의 개수  
    private int _addedChars ;          // 삽입된 문자의 개수  
  
    // 생성자  
    public AppController()  
    {  
        this._appView = new AppView() ;  
        this._inputChars = 0 ;  
        this._ignoredChars = 0 ;  
        this._addedChars = 0 ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```


□ Class "AppController" 의 구현 [2]

```
public class AppController {  
    // 생성자  
    ...  
  
    // 출력 관련  
    private void showAllFromBottom()  
    private void showAllFromTop()  
    private void showTopElement()  
    private void showStackSize ()
```

□ Class "AppController" 의 구현 [3] 스택

```
public class AppController {
```

```
...
```

```
// 횃수 계산
```

```
private void countAdded ()
```

```
private void countIgnored ()
```

```
private void countInputChar ()
```

```
// 스택 수행 관련
```

```
private void addToStack (char inputChar)
```

```
private void removeOne ()
```

```
private void removeN (int numberOfCharsToBeRemoved)
```

```
private void conclusion ()
```

□ Class "AppController" 의 구현 [4]

■ Private Member function의 구현

- private void `showAllFromBottom()` {
 // 스택의 모든 원소를 Bottom 부터 Top 까지 출력한다
 this.showMessage (MessageID.Notice_ShowStack) ;
 this.showMessage (MessageID.Show_StartBottom) ;
 for (int index = 0 ; index < this._arrayStack.size() ; index++) {
 this._appView.outputStackElement
 ((char) this._arrayStack.elementAt (index)) ;
 }
 this.showMessage (MessageID.Show_EndTop) ;
}
- private void `showAllFromTop()`
 - ◆ `_arrayStack` 의 `elementAt()` 을 이용하여 Top 부터 Bottom 까지 출력
 - ◆ `showAllFromBottom()` 을 참고하여 작성

□ Class "AppController" 의 구현 [4] 스택

■ Private Member function의 구현

- private void `showTopElement()`
 - ◆ `_arrayStack`의 `peek`을 이용하여 Top원소를 얻어와서 `appView`의 `outputTopElement`를 이용하여 출력
- private void `showStackSize()`
 - ◆ `_arrayStack`의 `size`을 이용하여 원소의 개수를 얻어와서 `appView`의 `outputStackSize`를 이용하여 출력

□ Class "AppController" 의 구현 [5]

■ private Member function의 구현

- private void `countAdded()`
 - ◆ `_addedChars`을 1 증가
- private void `countIgnored()`
 - ◆ `_ignoredChars`을 1 증가
- private void `countInputChar()`
 - ◆ `_inputChars`을 1 증가

□ Class "AppController" 의 구현 [6]

- public Member function의 구현
 - public void addToStack (char inputChar)

```
private void addToStack(char inputChar) {  
  
    //문자를 스택에 삽입한다  
  
    if(this._arrayStack.push(inputChar)) {  
        this._appView.outputAddedElement(inputChar);  
        this.countAdded();  
    }else{  
        this.showMessage(MessageID.Error_InputFull);  
    }  
}
```

□ Class "AppController" 의 구현 [7]

■ public Member function의 구현

- public void removeOne()

```
public void removeOne()
{
    // 문자 한개를 스택에서 삭제한다.
    if (this._arrayStack.isEmpty()) {
        this.showMessage(MessageID.Error_RemoveEmpty);
    }
    else {
        this._appView.outputRemove((char)this._arrayStack.pop());
    }
}
```

□ Class "AppController" 의 구현 [8]

■ Private Member function의 구현

- private void `removeN` (int numberOfCharsToBeRemoved)
 - ◆ `removeOne()` 을 원하는 숫자만큼 실행
- private void `conclusion` ()
 - ◆ `removeOne()` 을 사용하여 스택 내부에 있는 모든 원소를 pop 하면서 보여준다.
 - ◆ `appView` 의 `outputResult` 를 이용하여 입력된 문자, 정상 처리된 문자, 무시된 문자, 삽입된 문자의 개수 출력을 한다.

□ Class "AppController" 의 구현: run()

```
public void run()
{
    // 프로그램의 실질적인 실행을 위한 함수. 실질적인 main() 함수.
    this._arrayStack = new ArrayList<Character>();
    char input;

    this.showMessage(MessageID.Notice_StartProgram);
    this.showMessage(MessageID.Notice_StartMenu);
    input = this._appView.inputCharacter();
    while(input != '!') {
        this.countInputChar();
        if ((input >= 'A' && input <= 'Z') ||
            (input >= 'a' && input <= 'z')) {
            this.addToStack(input);
        }
        else if (input >= '0' && input <= '9') {
            this.removeN(input - '0');
        }
        else if (input == '-') {
            this.removeOne();
        }
        else if (input == '#') {
            this.showStackSize();
        }
    }
}
```

□ Class "AppController" 의 구현: run()

```
        else if (input == '/') {
            this.showAllFromBottom();
        }
        else if (input == '\\') {
            this.showAllFromTop();
        }
        else if (input == '^') {
            this.showTopElement();
        }
        else {
            this.showMessage(MessageID.Error_WrongMenu);
            this.countIgnored();
        }
        input = this._appView.inputCharacter();
    }
    this.showMessage(MessageID.Notice_EndMenu);
    this.conclusion();
    this.showMessage(MessageID.Notice_EndProgram);
}
```

Class "AppView"

□ AppView: 인스턴스 변수, 생성자

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 공개함수

■ 입력을 위한 공개 함수

- public int inputInt()
- public String inputString()
- public char inputCharacter()
 - ◆ Char 한 개만을 입력 받는 함수

```
public char inputCharacter() {  
    char element;  
    System.out.print("- 문자를 입력하시오: ");  
    element = this.inputString().charAt(0);  
    return element;  
}
```

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- `public void outputAddedElement (char anElement)`
- `public void outputStackElement (char anElement)`
- `public void outputTopElement (char anElement)`
- `public void outputStackSize (int aStackSize)`
- `public void outputRemove (char anElement)`
- `public void outputRemoveN (int numberOfCharsToBeRemoved)`
- `public void outputResult (int numberOfInputChars,
int numberOfIgnoredChars,
int numberOfAddedChars)`

- `public void outputMessage (String aMessageString)`

Enum "MessageID"

Enum "MessageID"

```
public enum MessageID {  
  
    // Message IDs for Notices:  
    Notice_StartProgram,  
    Notice_EndProgram,  
    Notice_StartMenu,  
    Notice_EndMenu,  
    Notice_InputStack,  
    Notice_DelStack,  
    Notice_ShowStack,  
  
    // Message IDs for Shows:  
    Show_StartBottom,  
    Show_StartTop,  
    Show_EndBottom,  
    Show_EndTop,  
  
    // Message IDs for Errors:  
    Error_WrongMenu,  
    Error_InputFull,  
    Error_RemoveEmpty,  
  
} //End of enum "MessageID"
```


Interface “Stack”

□ 인터페이스 (Interface)

■ Abstract Class 와 유사하다:

- Abstract method 의 정의로만 구성된다.
 - ◆ 당연히 구현 코드가 주어지지 않는다.
 - ◆ 함수의 header 만 정의된다.

■ 그렇지만, class 는 아니다:

- 상수나 인스턴스 변수를 선언할 수 없다.
- 생성자가 존재할 수 없다.
- 상속의 대상이 아니다:
 - ◆ "extends" 할 수 없다.

■ 사용법:

- "extends" 대신 "**implements**" 를 사용한다.
- 다중 "implements" 가 가능하다: 다중 상속?
 - ◆ "extends" 는 단 하나의 상속 class 만 허락한다.

□ Interface "Stack"

```
public interface Stack<T>
{
    public boolean isEmpty ();
    public boolean isFull () ;
    public boolean  push (T anElement) ;
    public T  pop () ;
    public T  peek () ;
}
```

스택을 구현하는 Class "ArrayList<T>"

ArrayList: 상수, 인스턴수 변수

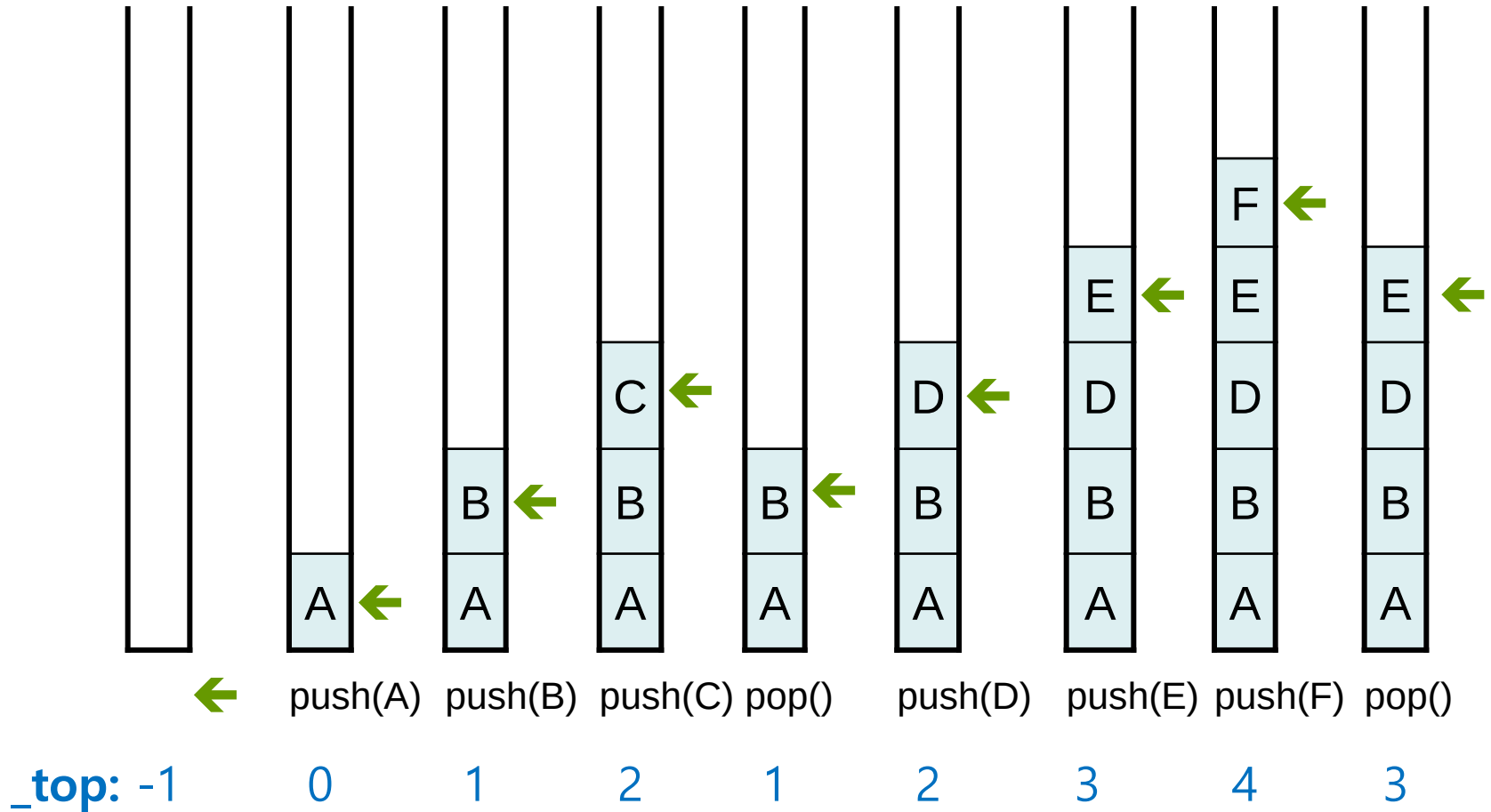
```
public class ArrayList<T> implements Stack<T> {  
    private static final int DEFAULT_CAPACITY = 5 ;  
    private int _capacity ;  
    private int _top ;  
    private T[] _elements ;
```

□ Member Functions의 구현

■ ArrayList의 Public Member function의 구현

- `public ArrayList ()`
- `public ArrayList (int givenCapacity)`
 - ◆ `this._capacity` 를 주어진 `givenCapacity` 로 초기화
 - ◆ `this._top`은 -1로 초기화
 - ◆ ArrayList의 원소가 T, 즉 generic 형태로 선언 되어 있으므로 `this._elements` 배열은 최상위 클래스 (가장 일반적인) Object 들의 배열로 생성
 - `this._elements =`
`(T[]) new Object [ArrayList.DEFAULT_CAPACITY] ;`
- `Public boolean isEmpty()`
 - ◆ `this._top`을 확인하여 비어있는지 확인
- `public boolean isFull()`
 - ◆ `this._top`을 확인하여 가득차 있는지 확인
- `public int size()`
 - ◆ `this._top`을 확인하여 삽입된 원소의 길이를 확인

□ 배열로 구현된 스택에서의 삽입/삭제



□ ArrayList: 공개함수 [1]

■ public boolean push (T anElement)

- 가득 차 있을 경우 false를 반환
 - ◆ Resize 는 고려하지 않는다.
- 가득 차 있지 않을 경우
 - ◆ this._top 을 1 증가
 - ◆ this._elements[] 의 this._top 위치에 anElement를 삽입

■ public T pop ()

- 비어 있으면 null을 반환
- 비어 있지 않으면
 - ◆ this._top 에 있는 _elements[] 의 원소를 반환
 - ◆ this._top 을 1 감소

□ ArrayList: 공개함수 [2]

■ public T peek()

- 비어 있으면 null 을 반환
- 비어 있지 않으면
 - ◆ this._element[] 의 this._top 에 있는 원소를 반환

■ public void clear()

- this._top 을 -1 로 초기화
- this._elements[] 의 모든 원소를 null 로 초기화

■ public T elementAt (int anOrder)

- 주어진 순서 (anOrder) 의 원소를 돌려준다.

요약과 정리



□ 정리

- Generic class 의 개념, 사용법
- Interface의 개념, 사용법
- Stack의 개념, 사용법

보고서 작성과 제출

□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제07주] 스택 -기본 기능
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

- DS1_07_학번_이름.zip

- 폴더의 구성

- ◆ DS1_07_학번_이름

- 프로그램

- 프로젝트 폴더 / 소스

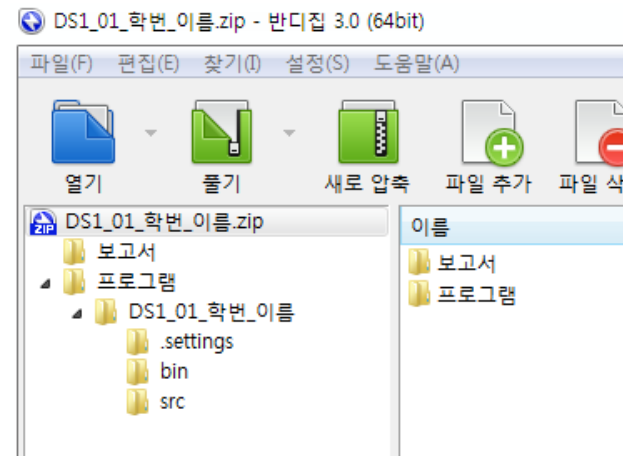
- 메인 클래스 이름 : DS1_06_학번_이름.java

- 보고서

- 이곳에 보고서 문서 파일을 저장한다.

- 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

- 문서는 pdf 파일로 만들어 제출한다.



[제 7 주] 실습 끝



